

Pitch Playbook

Aleph-1 · UnRiskOmega · 60-Second Client Briefing — per-slide script, technical notes & Q&A. Target ~5.5 min + live demo.

Slide 1 · Title

Purpose: One-line promise; who you are.

SAY *"We're Aleph-1. The case asked how an advisor preps for a client call fast. Our answer: a 60-second briefing that replaces a morning of clicking through dashboards."*

LIKELY QUESTIONS

Q What is it in one sentence?

A It turns a client's raw portfolio data into a grounded, 60-second briefing an advisor can trust before a meeting.

Slide 2 · The advisor problem

Purpose: Make them feel the pain before the fix.

SAY *"Before one conversation an advisor cross-checks 3 dashboards and 16 analysis tiles — about 15 minutes per client, and the one thing that matters is easy to miss."*

TECHNICAL — what it does / how we built it / what it solves

- Framing slide — no code. The 16 tiles are UnRiskOmega's own portfolio-dashboard panels; ~15 min is our manual-prep estimate.
- The real target it sets up: reduce prep time and lower the chance of missing a material risk (concentration, suitability breach, drawdown).

LIKELY QUESTIONS

Q Where do 16 tiles / 15 min come from?

A 16 = the analysis panels in the product's portfolio view; 15 min is our estimate of reading them per client. The ratio is the point, not the exact number.

Slide 3 · Trigger & user flow

Purpose: Show a product flow, not a one-off script.

SAY *"Four steps — open client, engine computes, AI narrates, advisor reads. The advisor never touches the raw tiles; the machine does the assembly."*

TECHNICAL — what it does / how we built it / what it solves

- Trigger = selecting a client (or a scheduled pre-meeting prompt in production).
- Under the hood the flow is: load_store() → build_dossier() (one client+portfolio) → build_briefing() (the facts) → app renders → AI writes the text.
- Solves: the advisor gets a conclusion, not a pile of dashboards to interpret.

LIKELY QUESTIONS

Q What's the trigger in production?

A Opening the client record, or a scheduled prompt before a booked meeting. In the demo it's the dropdown.

Slide 4 · The 60-second briefing + LIVE DEMO

Purpose: Prove it's real, on a client you've never seen.

SAY *"Three sections — situation, development, news. Let me run it on the client you just gave us." → pick client → Section 1 renders → click Generate → read one line → "Never-seen client, no config, same pipeline."*

TECHNICAL — what it does / how we built it / what it solves

- build_briefing(store, profile) returns ONE JSON dict with blocks: profile, performance, allocation, holdings, mandate, peers, projection, market_context, data_quality.
- Section 1 (situation) is pure deterministic display — no AI. Sections 2–3 add charts + on-demand AI text.
- The app caches the payload per client and calls the AI only on button-click, so a model/news hiccup never blanks the page.
- Solves: a readable, trustworthy briefing where every figure is computed, not generated.

LIKELY QUESTIONS

Q Are these numbers computed or written by the AI?

A Computed. Section 1 has no AI at all. Every figure traces to a source field; the AI only writes prose around numbers we calculated.

Q What if the briefing is wrong?

A It can't state a number we didn't compute, and if a check couldn't run it says 'not checked' rather than guessing.

Slide 5 · Data sources

Purpose: Answer 'where does this come from'.

SAY *"Two bank exports, live Apify news matched to holdings, and OpenAI for narration only — it never touches a number."*

TECHNICAL — what it does / how we built it / what it solves

- clients.json — 47 clients, 57 portfolios: holdings, accounts, transactions, suitability violations.
- reference.json — 504 securities, risk profiles, strategic allocations, rules, ~48k fund look-through rows.
- Apify news — queried per client's held instruments, not generic market noise.
- OpenAI (gpt-5-mini) — writes the 130–160 word text from computed facts. No account numbers or IBANs enter the payload.
- Solves: grounded inputs + a clean privacy boundary for the LLM.

LIKELY QUESTIONS

Q Is client data sent to OpenAI?

A Only the computed facts — no IBANs or account numbers. In production it'd be a governed LLM endpoint under the bank's controls.

Slide 6 · Analysis & recommendation logic

Purpose: The differentiator: anti-hallucination.

SAY *"The engine computes everything deterministically; the AI only describes and cites. Missing data stays 'not checked', never silently 'passed'. For a risk product, an AI that can't invent a number is the whole point."*

TECHNICAL — what it does / how we built it / what it solves

- Deterministic analytics modules compute: performance/risk, allocation drift, mandate guardrails, concentration, peers, projections.
- Grounding = 3 layers: AI receives only computed facts; it's instructed to describe not calculate; every number carries its source field, so it's verifiable.
- Missing-data model keeps four states distinct — absent / null / empty / invalid. 26 of 57 portfolios had no violation data; without this they'd all read as 'compliant'.
- Solves: trustworthy output in a regulated context; no fabricated compliance.

LIKELY QUESTIONS

Q How exactly do you prevent hallucination?

A AI gets only pre-computed facts, is told to describe not compute, and every payload number cites its source — so the text is checkable against the data.

Q What are the recommendations based on?

A Findings from the data — concentration, mandate drift, suitability — plus published screening thresholds (Citi 10%, J.P. Morgan 30%). They're flags for the advisor, not automated advice.

Slide 7 · Technical architecture

Purpose: Prove it's engineered, not glued together.

SAY *"Exports → a typed data foundation that preserves missing-data meaning → 8 analytics modules with 247 tests → one strict JSON payload → then the AI. Python, standard-library core, reproducible."*

TECHNICAL — what it does / how we built it / what it solves

- Data foundation (processing/): the loader parses raw clients.json keeping absent/null/empty/invalid distinct and never zero-filling; indexes join reference data by SecurityId (not ISIN); the dossier assembles one client+portfolio. Solves: trustworthy denominators and honest missing-data.
- Analytics engine (analytics/): 8 modules turn a dossier into computed metrics. Solves: deterministic facts, each traceable.
- Briefing payload: one strict JSON, every block has a status + reason, plus a data_quality section. Solves: a single contract the UI and AI both read.
- News module (news/): Apify queries built from held names, cached with a run budget; attaches as market_context. Solves: relevant, grounded news without re-fetching.

- AI narration (ai_engine.py): OpenAI call that only describes the payload and cites sources. Solves: fast, readable summary.
- App (app.py, Streamlit): 3 sections, payload cached, AI calls lazy. Solves: resilient UX.
- Cross-cutting: Python 3.11, stdlib core, 247 tests, reproducible (same input + seed → same output).

LIKELY QUESTIONS

Q Why is reproducibility a big deal?

A Same input and seed always yield the same briefing — essential when you must justify what the advisor was shown.

Q What's the missing-data handling?

A Four states kept distinct. It's why a portfolio with no violation data reads as 'not checked', never a false 'compliant'.

Slide 8 · Mock data & production

Purpose: Honesty about what's real vs what production needs.

SAY *"Every portfolio number is real, computed from the exports — nothing faked. Production swaps the static files for a live custodian feed, a licensed news API, and a governed LLM. The architecture already accepts a fresh client file with no code change."*

TECHNICAL — what it does / how we built it / what it solves

- Real today: all financial calculations from the provided exports; a real Apify news integration; a real OpenAI narration call.
- Stand-ins: the data SOURCE (static export vs live custodian/PMS feed); news needs an API key; the LLM would be a governed endpoint.
- Solves: nothing in the logic is faked — only the plumbing to production data differs.

LIKELY QUESTIONS

Q So what's actually mocked?

A Nothing in the financials. Only the data source (static file vs live feed) and the need for API keys. The logic is production-shaped.

Slide 9 · Completed bonus features

Purpose: Show range beyond the core.

SAY *"Holding-aware news, Monte Carlo projections, peer comparison, fund look-through — and we keep the full finding set so a follow-up chatbot can answer what the 60-second view didn't show."*

TECHNICAL — what it does / how we built it / what it solves

- Holding-aware news: queries built from the client's real positions; articles treated as untrusted data, never instructions.
- Monte Carlo: seeded bootstrap + parametric projections with a fan chart — labelled scenarios, not forecasts, shown next to the bank's own risk figures.
- Peer comparison: percentile vs a cohort, excluding consolidated views so nothing is double-counted.
- Fund look-through: funds decomposed into real industry / region / currency exposure.
- Chatbot-ready: the full finding collection is retained in the payload, so follow-up Q&A; has the material the briefing trimmed.

LIKELY QUESTIONS

Q You warned against Monte Carlo — why include it?

A We present it as a seeded scenario range, clearly labelled, alongside the bank's own risk figures — not as a forecast.

Q Is the chatbot built?

A The data layer for it is — the full findings are retained. The chat UI is the next step, not shipped.

Slide 10 · Close / unseen client

Purpose: Land the 'it generalises' point the brief explicitly tests.

SAY *"Drop in a client we've never seen — same briefing, no config. Same principle as the product: surface what matters, get to the point. We're Aleph-1, thank you."*

TECHNICAL — what it does / how we built it / what it solves

- build_briefing accepts any client ref or portfolio id; no per-client configuration or hard-coded example.
- We ran the full pipeline across all 47 clients / 57 portfolios — each produces a valid briefing, including the awkward shapes (cash-only, no risk profile, consolidated).
- Solves: the exact requirement that a fresh test client works, not just one preconfigured demo.

LIKELY QUESTIONS

Q How do you know it works on any client?

A We swept all 47 clients and 57 portfolios end-to-end — every one produces a valid briefing, including cash-only and consolidated portfolios.

Global Q&A — the hardest cross-cutting questions

Q Is that a real performance return?

A No — it's portfolio value change. The data has no cash flows, so we can't compute a GIPS return and never call it one; we label it precisely.

Q Can you show benchmark / alpha / beta?

A Not from this data — there's no benchmark in the reference set. We do peer comparison against a cohort instead.

Q Your data is months old — how is it 'current'?

A History ends July 2026. Every figure is dated 'as of' that point, never 'today' — we're strict about not implying freshness we don't have.

Q What does it cost to run?

A One LLM call per briefing (~150 words). News is cached with a run budget, so it isn't re-fetched on every refresh.

Q Why should a bank trust an AI here?

A Because the AI isn't doing the analysis — deterministic, tested code is. The AI only phrases it and can't state anything not in the data.

Q How do you handle a client with weird or missing data?

A The loader keeps absent/null/empty/invalid distinct and never zero-fills; checks that can't run are marked unavailable. We tested all 47 clients — it degrades gracefully, it doesn't crash.

Q How would this scale to thousands of clients?

A The store loads once and is reused; each briefing is independent and deterministic; Monte Carlo path count is tunable for speed.

Q What's the security story for client data?

A Account numbers and IBANs never enter the payload; only computed facts reach the LLM; production would use a governed endpoint under bank controls.

Demo safety net: keep a pre-generated briefing screenshot / short screen recording ready. If the live run stalls (wifi, API timeout), cut to it and keep talking — never a blank screen.